

LAB SESSION 4

Lab Name: Software Requirement Specification (SRS)

Project: MediScript-E — Digital Healthcare Platform

Objectives

- Prepare a formal IEEE-standard SRS document for MediScript-E
 - Learn requirement documentation best practices
 - Establish a contract between stakeholders and developers
-

Theory

The SRS document acts as:

- **A Contract:** Agreement between stakeholders and development team
- **A Design Reference:** Blueprint for system architecture and implementation
- **A Testing Baseline:** Foundation for test case design and validation

A good SRS must be **complete**, **unambiguous**, and **verifiable**. For MediScript-E, the SRS ensures all healthcare workflows are properly documented before implementation.

Software Requirement Specification Document

1. Introduction

1.1 Purpose

This SRS document defines the functional and non-functional requirements for **MediScript-E**, a digital healthcare platform. It serves as a reference for developers, testers, and stakeholders throughout the software development lifecycle.

1.2 Scope

MediScript-E is a web-based healthcare platform that enables:

- Patients to book appointments, manage prescriptions, set medicine reminders, and store medical records
- Doctors to manage appointments, issue digital prescriptions, and archive/edit prescriptions
- Administrators to monitor platform activity and manage users
- All users to interact with an AI-powered chatbot (MediBot) for platform assistance
- All authenticated users to use a global search feature

The system is built using **Next.js 16**, **PostgreSQL (Supabase)**, **Prisma ORM**, **Groq AI SDK**, and deployed on **Vercel**.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirement Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
OTP	One-Time Password
2FA	Two-Factor Authentication
JWT	JSON Web Token
API	Application Programming Interface
PDF	Portable Document Format
OAuth	Open Authorization
AI	Artificial Intelligence
LLM	Large Language Model

1.4 References

- Next.js 16 Documentation: <https://nextjs.org>
- Prisma ORM Documentation: <https://prisma.io>
- Supabase Documentation: <https://supabase.com>
- NextAuth.js Documentation: <https://next-auth.js.org>
- Groq API Documentation: <https://console.groq.com>
- IEEE Std 830-1998: IEEE Recommended Practice for SRS

1.5 Overview

This document is organized as follows:

- Section 2: Overall system description
- Section 3: Functional requirements
- Section 4: Non-functional requirements
- Section 5: Constraints and assumptions

2. Overall Description

2.1 Product Perspective

MediScript-E is a standalone web application that integrates with:

- **Supabase** for PostgreSQL database and file storage
- **Google & GitHub OAuth** for third-party authentication with profile pictures
- **Gmail SMTP (Nodemailer)** for email notifications
- **Groq AI API** for AI chatbot (MediBot) powered by Llama 3.1
- **Vercel** for cloud deployment

2.2 Product Functions

The system provides the following major functions:

1. User authentication and authorization (email/password, OAuth, 2FA)
2. Appointment booking and management with dedicated pages
3. Digital prescription issuance, PDF download, archive, and edit
4. Medicine reminder scheduling with email alerts
5. Medical vault for secure document storage
6. Admin dashboard with real-time statistics
7. AI-powered chatbot (MediBot) for platform assistance
8. Global search across appointments, prescriptions, and users
9. Profile picture display from OAuth providers

2.3 User Classes and Characteristics

User Class	Description	Technical Level
Patient	Registers, books appointments, manages health records	Low to Medium
Doctor	Manages appointments, issues and archives prescriptions	Medium
Admin	Monitors platform, manages users	High
Public	Views landing page, submits contact form, uses chatbot	Low

2.4 Operating Environment

- **Client:** Modern web browsers (Chrome, Firefox, Safari, Edge)
- **Server:** Vercel Serverless Functions (Node.js runtime)
- **Database:** PostgreSQL hosted on Supabase
- **Storage:** Supabase Storage for medical documents
- **AI:** Groq API (Llama 3.1 8B Instant model)

2.5 Design and Implementation Constraints

- Must use Next.js App Router architecture with dedicated routes per feature
- Must use Prisma ORM for all database operations
- Must comply with Vercel serverless function limitations
- File uploads limited to supported medical document formats
- AI chatbot must only answer platform-related questions

2.6 Assumptions and Dependencies

- Users have access to a valid email address
- Users have a stable internet connection
- Supabase services remain available
- Gmail SMTP credentials remain valid for email delivery
- Groq API key remains valid for AI chatbot responses

3. Functional Requirements

3.1 Authentication Module

FR ID	Requirement	Priority
FR-01	System shall allow users to register with name, email, password, and role (PATIENT/DOCTOR)	High
FR-02	System shall send an email verification link upon successful registration	High
FR-03	System shall prevent login until email is verified	High
FR-04	System shall allow verified users to login with email and password	High
FR-05	System shall support OAuth login via Google and GitHub	Medium
FR-06	System shall auto-verify OAuth users and create a patient profile	Medium
FR-07	System shall display OAuth profile pictures in navbar and dashboard sidebar	Medium
FR-08	System shall allow users to enable 2FA email OTP from settings	High
FR-09	System shall send a 6-digit OTP to user email when 2FA is enabled during login	High
FR-10	System shall invalidate OTP after 10 minutes or successful use	High
FR-11	System shall allow users to resend OTP with a 60-second cooldown	Medium

3.2 Patient Module

FR ID	Requirement	Priority
FR-12	System shall provide a dedicated /appointments page for appointment management	High
FR-13	System shall display list of available doctors for appointment booking	High
FR-14	System shall allow patients to book appointments with date, time, and reason	High
FR-15	System shall allow patients to view all their appointments with status filters	High
FR-16	System shall allow patients to cancel pending appointments	Medium
FR-17	System shall provide a dedicated /prescriptions page for prescription management	High
FR-18	System shall allow patients to view prescriptions issued by doctors	High
FR-19	System shall allow patients to download prescriptions as PDF	High
FR-20	System shall provide a dedicated /reminders page for medicine reminders	Medium
FR-21	System shall allow patients to add medicine reminders with name, dosage, frequency, and schedule	Medium
FR-22	System shall send automated email alerts for medicine reminders via cron job	Medium
FR-23	System shall allow patients to mark medicine as taken or undo	Medium
FR-24	System shall provide a dedicated /vault page for medical vault	Medium
FR-25	System shall allow patients to upload medical documents to Medical Vault	Medium
FR-26	System shall allow patients to delete uploaded medical documents	Low
FR-27	System shall display patient blood group selected during registration	Low

3.3 Doctor Module

FR ID	Requirement	Priority
FR-28	System shall display all appointments assigned to the doctor with filter tabs	High
FR-29	System shall allow doctors to confirm pending appointments	High
FR-30	System shall allow doctors to cancel appointments with confirmation dialog	High
FR-31	System shall allow doctors to mark appointments as completed with confirmation dialog	High
FR-32	System shall allow doctors to issue prescriptions with patient selection, diagnosis, and medications	High
FR-33	System shall automatically mark appointment as COMPLETED when prescription is issued	High
FR-34	System shall allow doctors to edit issued prescriptions	Medium
FR-35	System shall allow doctors to delete issued prescriptions	Medium
FR-36	System shall allow doctors to archive and unarchive prescriptions	Medium
FR-37	System shall display Active and Archived prescription tabs for doctors	Medium

3.4 Admin Module

FR ID	Requirement	Priority
FR-38	System shall display real-time statistics on /dashboard overview page	High
FR-39	System shall provide a dedicated /users page for user management	High
FR-40	System shall allow admin to view all registered users with their profiles	High
FR-41	System shall allow admin to delete any user except themselves	High
FR-42	System shall provide a dedicated /appointments page for all appointments	Medium
FR-43	System shall allow admin to view all appointments with status filter	Medium
FR-44	System shall provide a dedicated /contacts page for contact messages	Low
FR-45	System shall allow admin to view all contact form submissions	Low

3.5 AI Chatbot Module

FR ID	Requirement	Priority
FR-46	System shall provide a floating AI chatbot (MediBot) accessible on all pages	Medium
FR-47	MediBot shall answer questions about MediScript-E features and usage	Medium
FR-48	MediBot shall always respond in English with plain conversational text	Medium
FR-49	MediBot shall maintain conversation history within a session	Medium
FR-50	MediBot shall redirect unrelated questions back to platform topics	Medium

3.6 Search Module

FR ID	Requirement	Priority
FR-51	System shall provide a global search bar in the dashboard header	Medium
FR-52	Search results shall be role-based (Patient: appointments/prescriptions, Doctor: patients/appointments, Admin: users/appointments)	Medium
FR-53	Search shall use 350ms debounce to avoid excessive API calls	Low
FR-54	Clicking a search result shall scroll to the relevant section	Low

3.7 General Module

FR ID	Requirement	Priority
FR-55	System shall allow users to update their display name	Medium
FR-56	System shall allow users to change their password with current password verification	Medium
FR-57	System shall allow public users to submit a contact form	Low
FR-58	System shall provide smooth page transition animations in dashboard	Low
FR-59	Dashboard sidebar shall highlight the active page/route	Medium

4. Non-Functional Requirements

4.1 Performance

- **NFR-01:** Pages shall load within 3 seconds under normal network conditions
- **NFR-02:** API responses shall complete within 2 seconds for standard database queries
- **NFR-03:** PDF generation shall complete within 5 seconds
- **NFR-04:** AI chatbot responses shall complete within 3 seconds

4.2 Availability

- **NFR-05:** System shall maintain 99.9% uptime in production environment
- **NFR-06:** Scheduled cron jobs for medicine reminders shall execute with less than 5-minute delay

4.3 Security

- **NFR-07:** All passwords shall be hashed using bcryptjs with minimum 10 salt rounds
- **NFR-08:** All database connections shall use SSL/TLS encryption
- **NFR-09:** JWT sessions shall expire after 30 days of inactivity
- **NFR-10:** Email verification tokens shall expire within 24 hours
- **NFR-11:** 2FA OTP codes shall expire within 10 minutes
- **NFR-12:** All API routes shall implement role-based access control (RBAC)
- **NFR-13:** Medical file uploads shall be validated for file type and size
- **NFR-14:** All user inputs shall be validated and sanitized before processing
- **NFR-15:** Search results shall only return data belonging to the authenticated user's role

4.4 Usability

- **NFR-16:** System shall be fully responsive across mobile, tablet, and desktop devices
- **NFR-17:** System shall provide clear success and error feedback for all user actions
- **NFR-18:** System shall support smooth page transitions and animations in dashboard
- **NFR-19:** Dashboard sidebar shall clearly indicate the currently active page
- **NFR-20:** AI chatbot shall be accessible via a floating button on all pages

4.5 Maintainability

- **NFR-21:** System codebase shall follow TypeScript strict typing standards
- **NFR-22:** All database operations shall use Prisma ORM for consistency
- **NFR-23:** System shall follow Next.js App Router conventions with dedicated routes

4.6 Scalability

- **NFR-24:** System shall be deployable on Vercel with zero-downtime deployments
- **NFR-25:** Database connection pooling shall be configured for concurrent users

5. Constraints & Assumptions

5.1 Constraints

- System must be deployed on Vercel platform
- Database must use PostgreSQL via Supabase
- Email notifications require valid Gmail SMTP credentials
- OAuth login requires valid Google and GitHub OAuth application credentials
- Cron job for medicine reminders requires a valid CRON_API_KEY
- File storage requires a configured Supabase storage bucket named **medical-reports**
- AI chatbot requires a valid Groq API key

5.2 Assumptions

- All users have access to a valid and active email address
 - Doctors provide valid medical license numbers during registration
 - Admin account is pre-seeded in the database
 - Users have access to modern web browsers with JavaScript enabled
 - Internet connectivity is available for all users during system use
 - Groq API service remains available for AI chatbot functionality
-

Key Findings / Learning Outcomes

- Updated SRS to include **59 Functional Requirements** across 7 modules and **25 Non-Functional Requirements** across 6 categories
- New modules added: AI Chatbot (MediBot), Global Search, dedicated feature pages
- Understood how SRS evolves as new features are added during development
- Recognized that professional web apps require dedicated routes per feature rather than single-page scroll navigation
- AI integration introduces new NFRs around response time and content safety